

JODL Multi-AI Orchestration Sy

A production-grade, filesystem-based
providers — Claude (Anthropic), Gem
atomic queue. No database, no daem

3

AI PROVIDERS

20+

AGENT ROLES

3

OR

∞

PARALLEL SESSIONS

01 What is this system?

JODL is a multi-AI co-work orchestration system. You submit a brief (e.g. "build a luxury fashion store with RAG-powered search"). The system automatically routes sub-tasks to the best AI provider for each domain, runs them in dependency order (or in parallel), and collects outputs.

Core insight: Different AI providers excel at different domains. Gemini has 2M context and multimodal input — ideal for design research. Claude has the best reasoning for architecture decisions. GPT / Codex runs directly in the codebase. By routing tasks to the right provider, you get better outcomes AND more efficient context usage than using one model for everything.

Three Providers

Claude (claude-code), Gemini (antigravity), GPT (codex) — each runs on its own platform but shares the same task queue via the filesystem.

Zero Infrastructure

The queue is literally files on disk. Atomic `rename()` is the distributed lock. Works across Windows, Mac, Linux, WSL — no DB, no server.

Intelligence Layer

A shared component registry, graph of pairings, feedback loop, and pattern memory accelerates every agent. Built knowledge survives sessions.

What's proven (working now)

- Full command bus: brief → session → task lifecycle
- Atomic claim (race-free across concurrent providers)

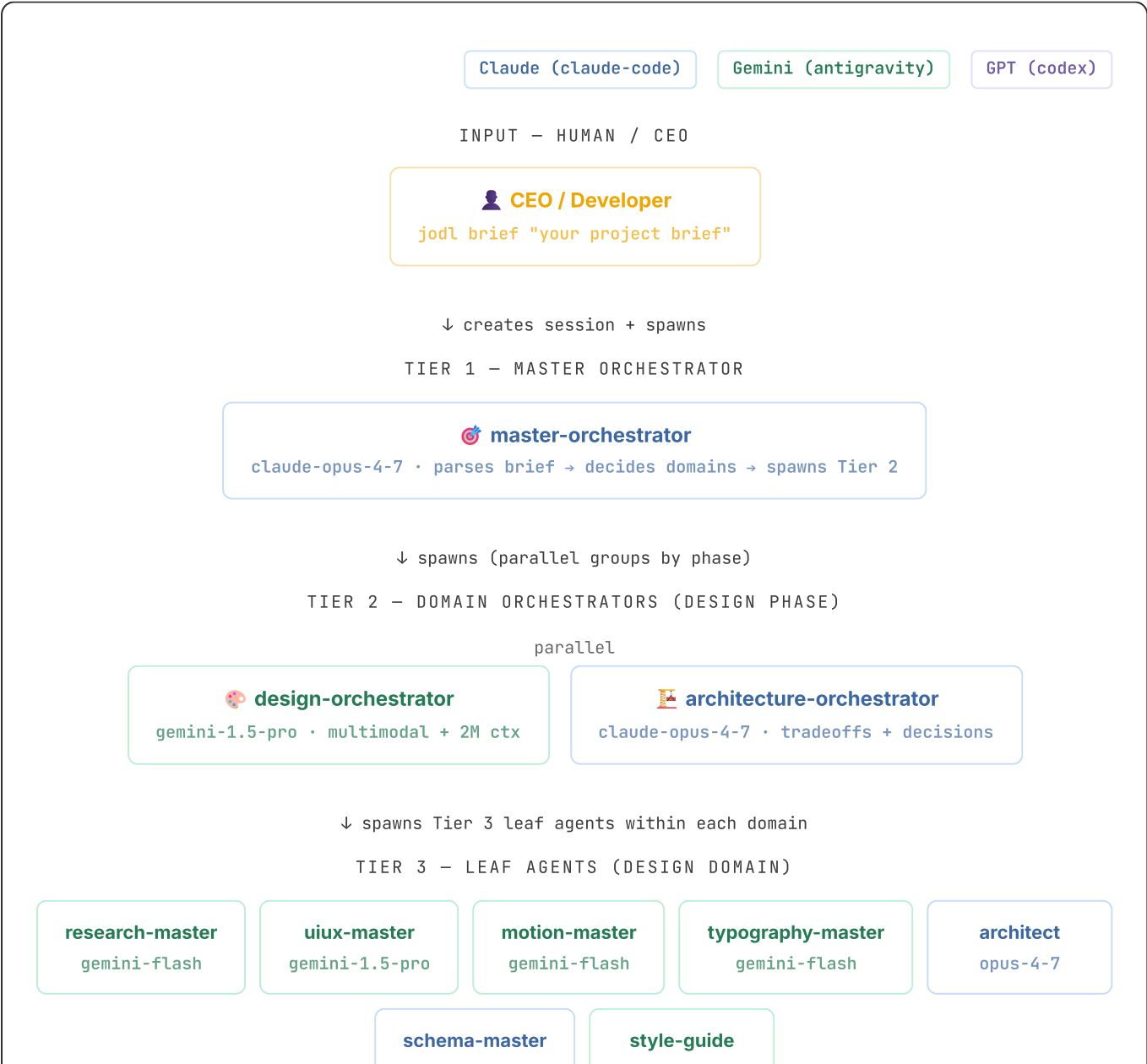
What still needs work

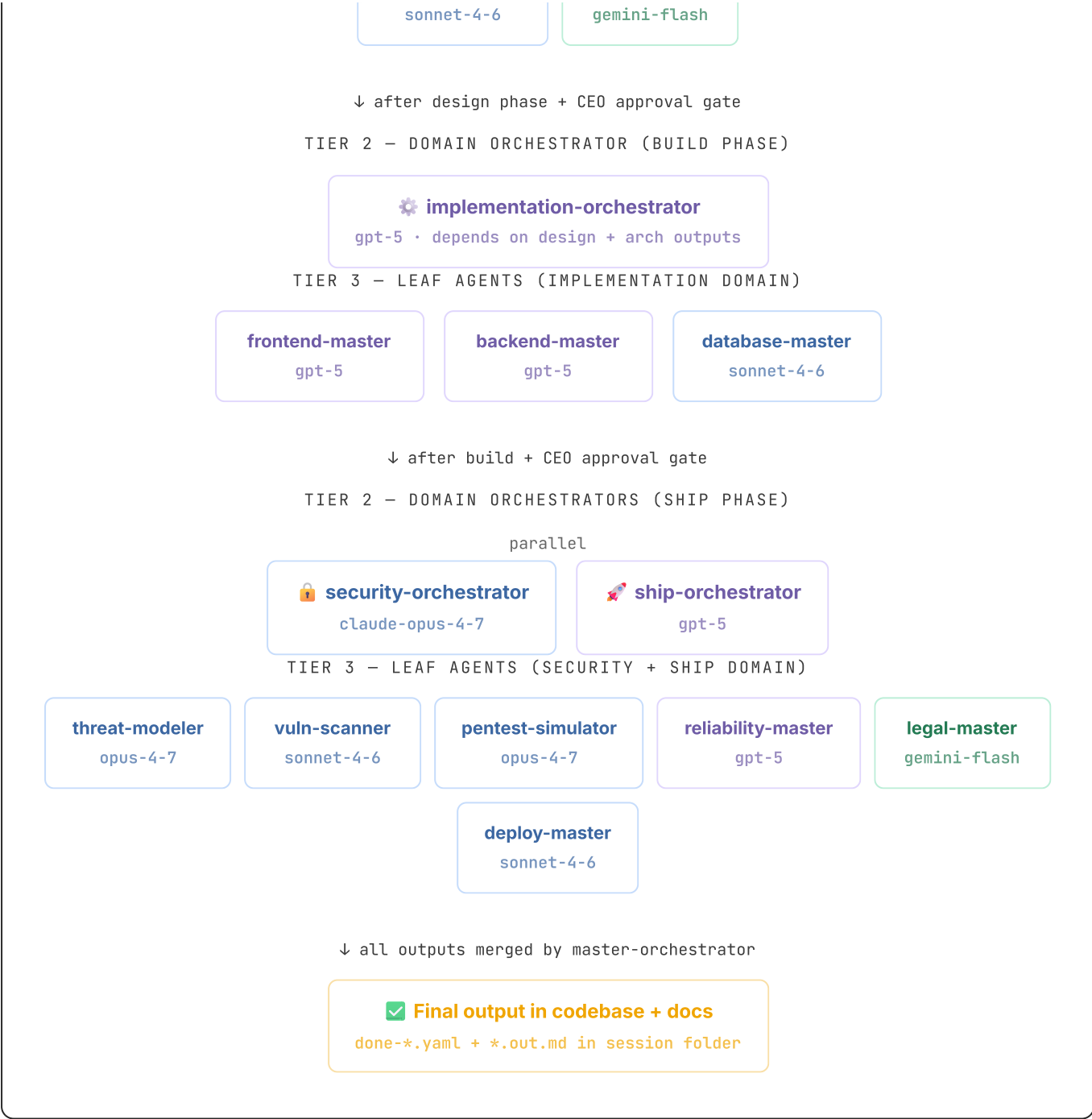
- System prompts for orchestrator roles (only leaf agents have them)
- Gemini model names need verification vs Google API

- Spawn-chain: orchestrators create child tasks from JSON
 - Claude auto-execution via `jodl next`
 - Manual flow (claim + submit) for Gemini/GPT
 - Routing matrix YAML → role→provider mapping
 - Session context injection to every agent
 - Proven run: design spec → full Zara-style store implementation
 - Multi-provider execution (Sprint 1 shipped today)
 - Domain-scoped context (Sprint 1 shipped today)
- Pattern registry: 5 components, needs 30+
 - No embeddings yet (sqlite-vss C++ blocked on Windows)
 - Parallel group execution not fully automated
 - Web dashboard (currently terminal watch command only)
 - creat-studio + jodlxverse apps not scaffolded
 - Knowledge packs missing for most agents

02 3-Tier Orchestration Architecture

Every session follows a strict hierarchy. Tasks are created by orchestrators, consumed by the next tier, and chained via a dependency graph. Parallel groups run concurrently when both providers are active.





Routing is declarative. The routing matrix at `D:\.agents\routing-matrix.yaml` maps every role to a provider+model. Change one line to redirect any agent to a different model. The bus reads this on every task dispatch.

03 Command Bus — The Core Mechanism

The command bus is the backbone of the system. It's a lockless, filesystem-based distributed task queue. No Redis, no Postgres, no broker — just files and an atomic OS rename.

3.1 — The Atomic Lock Pattern

Key insight: On any OS (Windows NTFS, Linux ext4, macOS APFS), `rename(A → B)` is atomic within the same volume. If two processes try to rename the same file, exactly one wins. This is the distributed lock — no DB transaction needed.



Race condition handled: If two providers both attempt `rename(pending-X → claimed-provider-X)` simultaneously, exactly one succeeds (OS guarantee). The loser gets an ENOENT error and retries with the next task.

3.2 — Task YAML Anatomy

```

# D:\.agents\command-bus\active\sess-87af32f2\tasks\pending-7f5fec23.yaml id: 7f5fec23
sessionId: sess-87af32f2 role: frontend-master assignedProvider: codex # who must run this
assignedModel: gpt-5 phase: build dependsOn: [a9f2c1ca] # must finish first parallelGroup: 1
# runs parallel with group 1 brief: | Implement the Zara-style Header, Shop, ProductCard,
ProductPage... [full task description – can be hundreds of lines]
  
```

3.3 — Spawn Chain (Orchestrator → Children)

Every orchestrator's output must end with a JSON block. The bus parses this block and automatically creates child task YAML files. This is how work cascades through the hierarchy.

```

# Orchestrator output ends with this block (``json): { "phase": "build", "spawn-tasks": [ {
"role": "frontend-master", "brief": "Implement the Store page with Zara-style layout...",
"depends-on": [], "parallel-group": 1 }, { "role": "backend-master", "brief": "Build FastAPI
endpoints for products, cart, auth...", "depends-on": [], "parallel-group": 1 # same group =
runs in parallel } ] }
  
```

- master-orchestrator (task a9f2c1ca) – outputs JSON spawn block

↳ design-orchestrator
antigravity · parallel-group 1

↳ architecture-orchestrator
claude-code · parallel-group 1

↳ implementation-orchestrator
codex · depends-on [design, arch]

↳ security-orchestrator
claude-code · phase ship

↳ ship-orchestrator
codex · phase ship

3.4 — Dependency Resolution

`findNextTask(provider)` in `bus.ts` only returns a task if: (1) status is `pending`, (2) `assignedProvider` matches current env, (3) ALL ids in `dependsOn` are in the `done` set. This means parallel groups execute automatically as their dependencies complete.

```
# How deps work: design-orchestrator - dependsOn: [] runs immediately
architecture-orchestrator - dependsOn: [] runs immediately (parallel)
implementation-orchestrator - dependsOn: [design, arch] waits for both
security-orchestrator - dependsOn: [implementation] waits + phase=ship gate
```

04 End-to-End Pipeline Flow

A complete session from CEO brief to deployed code. Gates are points where a human must approve before the next phase unlocks.

1	CEO submits brief Human writes a project brief: "Build X with Y using Z." Session folder created. Master-orchestrator task file written to disk. Session ID generated. jodl brief "Build a luxury fashion store with real-time AI search" claude-code	INPUT
2	Master Orchestrator runs Claude Opus reads the brief. Decides which domains are needed. Outputs a JSON spawn-tasks block. Bus parses it and creates domain orchestrator task files — one per domain. Sets parallel groups and depends-on chains. jodl next (as claude-code) claude-code	DESIGN
3	Design + Architecture orchestrators run in parallel Gemini (design-orchestrator) analyzes visual references, generates design system, typography, motion spec. Claude (architecture-orchestrator) decides tech stack, data models, API contracts. Both run simultaneously — neither waits for the other. jodl next (on each provider's terminal) antigravity claude-code	DESIGN PARALLEL
4	Domain leaf agents run within each orchestrator's domain Each domain orchestrator's spawn-tasks creates leaf tasks: research-master, uiux-master, motion-master (design domain). architect, schema-master (arch domain). Each gets scoped context — only sees outputs from their own domain + orchestrator summaries. jodl next (loops until all tasks in domain done) antigravity claude-code	DESIGN
— GATE: CEO reviews design outputs — approves before Build phase unlocks		
5	Implementation Orchestrator runs GPT reads ALL design + architecture outputs. Creates a concrete implementation plan. Spawns frontend-master, backend-master, database-master tasks. Has access to the actual codebase via Codex CLI. jodl next (as codex) codex	BUILD
6	Implementation leaf agents write actual code frontend-master writes React components. backend-master writes API routes. database-master writes schema migrations. All agents write to the real codebase. Outputs are code diffs / file paths changed. codex claude-code	BUILD

	jodl next (each provider loops)	
🛑 GATE: CEO reviews code – approves before Ship phase unlocks (manual trigger)		
7	Security + Ship orchestrators run in parallel Security orchestrator (Claude) runs threat modeling, vuln scanning, pentest simulation. Ship orchestrator (GPT) handles deploy config, reliability checks, legal review. Both run simultaneously. jodl next (on each provider) claude-codecodex	SHIP PARALLEL
8	Master Orchestrator merges + final report Master orchestrator reads all done outputs across all domains. Writes a final summary: what was built, decisions made, tests to run, deployment steps. Session moves to done/. jodl next (final merge task) claude-code	DONE

05 Provider Profiles & Routing

Each provider is an AI platform with a unique identity, strengths, and API. The routing matrix at `D:\.agents\routing-matrix.yaml` is the single source of truth for which provider runs what.



claude-code

ANTHROPIC_API_KEY

Best for: reasoning, architecture, security. Runs autonomously via jodl next with Anthropic streaming SDK.

master-orchestrator	opus-4-7
architecture-orchestrator	opus-4-7
security-orchestrator	opus-4-7
architect	opus-4-7
schema-master	sonnet-4-6
threat-modeler	opus-4-7
vuln-scanner	sonnet-4-6
deploy-master	sonnet-4-6



antigravity

GOOGLE_API_KEY

Best for: design, research, multimodal, 2M context. Runs via @google/genai SDK (auto-execution added in Sprint 1).

design-orchestrator	gemin-1.5-pro
research-master	gemin-1.5-flash
uiux-master	gemin-1.5-pro
motion-master	gemin-3.5-flash
typography-master	gemin-3.5-flash
style-guide	gemin-3.5-flash
legal-master	gemin-3.5-flash



codex

OPENAI_API_KEY

Best for: code generation, implementation, deploys. Runs in real codebase via Codex CLI + OpenAI SDK (Sprint 1).

implementation-orchestrator	gpt-5
frontend-master	gpt-5
backend-master	gpt-5
reliability-master	gpt-5
ship-orchestrator	gpt-5

Override Rules

The routing matrix supports keyword-based overrides. When a brief contains certain keywords, specific roles are force-reassigned or skipped:

KEYWORD IN BRIEF	EFFECT
"quick prototype"	Downgrades research + uiux to flash models, skips security + architecture orchestrators
"production"	Upgrades legal-master to claude-opus for stricter analysis
"fix bug"	Skips design + research, runs only implementation-orchestrator

06 Context Injection & Domain Scoping

Every agent gets a system prompt + the outputs of all previously completed tasks injected as context. Before Sprint 1, ALL outputs were injected into ALL agents. This wasted tokens on unrelated work. The new domain scoping keeps context tight.

How it works: `domainForRole(role)` derives the domain from the role name.

`loadSessionContext(sessionId, domain)` filters done outputs by domain. No new YAML fields required — domain is inferred.

What each agent sees:

AGENT TYPE	DOMAIN	SEES	DOESN'T SEE
master-orchestrator	meta	ALL outputs from all domains	—
design-orchestrator	meta	ALL outputs	—
uiux-master	design	design domain outputs + all orchestrators	implementation, security, ship outputs
frontend-master	implementation	implementation outputs + all orchestrators	design leaf outputs, security leaf outputs
threat-modeler	security	security outputs + all orchestrators	design, implementation leaf outputs

Why this matters: A 10-task session with full outputs might be 40,000 tokens. With domain scoping, a leaf agent in the design domain might only see 8,000 tokens of relevant context. This keeps costs down and reduces hallucinations from unrelated noise.

Context injection path:

```
# What ends up in the agent's system prompt: [system-prompt.md content] ← role-specific behavior --- ## Prior session outputs ## master-orchestrator (task a9f2c1ca) Brief: Design a Zara-style luxury store... [orchestrator output – always included] ## uiux-master (task
```

b2c3d4e5) Brief: Design the product card component... [same-domain output – included for design agents] # ↑ implementation agents do NOT see this ↑

07 jodl-system — The Intelligence Layer

Parallel to the command bus, the `jodl-system` package provides a growing knowledge base that agents can query. This is what makes the system smarter over time — it learns from every project.

7	Feedback Loop graph/feedback.json Every component gets a 1-5 rating after use. Auto-deprecation if avg < 2 after 3+ uses. Ratings flow back to search scoring.
6	Agent Knowledge Packs agents/*/system-prompt.md Per-role system prompts + optional knowledge-pack.md with domain expertise. Currently: uiux, motion, typography, frontend, backend have prompts. Orchestrators use fallback generic prompt.
5	Embedding Index embedding index MISSING Planned: vector embeddings for semantic component search. Blocked on sqlite-vss (C++ build on Windows). Alternative: OpenAI/Anthropic embedding API + JSONL vectors file.
4	Search (Phase 1) src/search.ts Keyword search: tag match (3× weight) + name match (2×) + quality bonus. <code>compose(brief)</code> combines search results with graph pairings for full page composition context.
3	Component Graph graph/pairings.json Directed graph of which components pair well together. e.g. PreloaderScreen → CartDrawer → ProductCard. Used by <code>compose()</code> to suggest full page sets.
2	Pattern Registry registry/components.json 5 components currently (needs 30+). Each entry: name, description, tags, quality, confidence, source-app. Currently all from SARTA. Needs components from every project.
1	Source Harvest sources/ Raw harvested reference sites via <code>jodl curate <url></code>. Playwright-based screenshot + metadata extraction. Template in <code>sources/.template/</code>. Currently empty.

Pattern Promotion Path

A component's journey through the intelligence system: source/site-name/ ← jodl curate https://example.com ↓ manual extraction registry/components.json ← confidence: staging ↓ used in 2+ projects, rated ≥ 4 avg registry/components.json ← confidence: proven (auto-search-boosted) ↓ deprecated: avg < 2 after 3 uses registry/components.json ← confidence: deprecated (hidden from results)

D:\.agents is a git-versioned memory store shared across ALL AI models. Every mistake, proven pattern, and experiment is recorded here. This is what survives between sessions and prevents repeated mistakes.

✔ Confidence Tiers

- experiments/ — bold untested. Write BEFORE making the move.
- staging/ — learned once, not proven. Write during session.
- patterns/ — proven 2+ sessions or projects. Graduate from staging.
- mistakes/ — confirmed failures. Never repeat. 1 occurrence is enough.

🔑 Integrity Rules

- ALWAYS git commit D:\.agents after any write.
- Never delete entries — set `confidence: deprecated`.
- Wrong entry: `git log D:\.agents → git revert <sha>`.
- Every entry requires frontmatter: confidence, date, project, sessions-validated.

Current Brain Contents

TIER	ENTRY	WHAT IT RECORDS
MISTAKE	gstack-fullstack-miss	Full-stack request → missed gstack skill routing
MISTAKE	baseUrl-deprecated-ts6	TS6 bundler mode auto-infers baseUrl, needs ignoreDeprecations
MISTAKE	codex-react-anti-patterns	11 confirmed Codex output bugs that always need manual review
PATTERN	vite-ts-path-aliases	@/ alias requires both vite.config.ts AND tsconfig.app.json
PATTERN	feature-based-architecture	features/ + components/layout/ + components/ui/ + lib/ + types/ + data/
PATTERN	products-type-extraction	Extract Product type to types/ then re-export from data/ for compat
PATTERN	jodl-forge-phase0-scan	Always scan package.json + src/ before routing any skill
PATTERN	model-routing-signals	Emit [MODEL] ↑/↓ tag when task complexity changes
STAGING	lenis-version-windows-sqlite	lenis@^1.3.0 not 2.0; better-sqlite3 needs C++ build tools
STAGING	jodl-bus-multi-provider	runClaude/Gemini/OpenAI + domain context scoping + pnpm allowBuilds fix

09 File System Structure

Two root directories form the complete system. `D:\.agents` is the shared brain + bus. `D:\jodl-workspace` is the monorepo with all apps + the CLI.

D:\.agents — Shared Brain + Command Bus

```
D:\.agents\ |— ✓ BRAIN.md # master index – loaded every session |— ✓ routing-matrix.yaml
# maps roles → providers + models |— ✓ jodl-runbook.md # protocol doc for all AI
providers |— ✓ patterns\ |— vite-ts-path-aliases.md |— feature-based-
architecture.md |— products-type-extraction.md |— jodl-forge-phase0-scan.md |—
model-routing-signals.md |— ✓ mistakes\ |— gstack-fullstack-miss.md |— baseurl-
deprecated-ts6.md |— codex-react-implementation-patterns.md |— ~ staging\ |— lenis-
version-windows-sqlite.md |— jodl-bus-multi-provider.md # Sprint 1 just added |—
experiments\ # bold moves – write before the move |— ✓ models\ |— capabilities.md #
model routing table |— ✓ projects\ |— SARTA.md |— jodl-workspace.md |— ...others
|— ✓ handoffs\ |— 2026-05-27-sarta-redesign-antigravity.md |— ✓ provider-profiles\
|— claude-code.yaml |— antigravity.yaml |— codex.yaml |— ✓ command-bus\ |—
active\ |— ✓ sess-87af32f2\ # proven session |— brief.md |— session.json |—
tasks\ |— done-a9f2c1ca.yaml # design task done |— a9f2c1ca.out.md # design spec
output |— done-7f5fec23.yaml # implementation done |— 7f5fec23.out.md # full Zara
store impl output |— done\ # completed sessions archived here
```

D:\jodl-workspace — Monorepo

```
D:\jodl-workspace\ |— ✓ pnpm-workspace.yaml # workspace packages + allowBuilds |— ✓
turbo.json # build pipeline (build→typecheck→test) |— ✓ tools\jodl-cli\ # THE CLI –
jodl command entry point |— src\ | |— ✓ bus.ts # core: sessions, tasks, lock, context
| |— ✓ cli.ts # commander entry, registers all commands | |— commands\ | |— ✓ bus-
commands.ts # whoami, brief, next, status, claim, submit | |— ✓ agent.ts # direct agent
execution (non-bus) | |— ✓ watch.ts # live terminal dashboard | |— curate.ts # jodl
curate: harvest reference sites | |— embed.ts # rebuild embedding index | |— vendor.ts
# extract app as standalone |— package.json # deps: @anthropic-ai/sdk, @google/genai,
openai, chalk, commander |— ✓ packages\jodl-system\ # intelligence layer |— src\ | |
|— ✓ search.ts # Phase 1 keyword search + compose() | |— ✓ registry.ts # load/save
components.json | |— ✓ graph.ts # pairings graph traversal | |— ✓ feedback.ts #
rating + auto-deprecate |— ✓ agents\ | |— ✓ uiux-master\ system-prompt.md +
knowledge-pack.md | |— ✓ frontend-master\ system-prompt.md | |— ✓ motion-master\
system-prompt.md | |— ✓ typography-master\ system-prompt.md | |— ✓ backend-master\
system-prompt.md | |— X master-orchestrator\ MISSING – uses generic fallback | |— X
design-orchestrator\ MISSING – uses generic fallback |— registry\ |— ~ components.json
5 components (needs 30+) |— ✓ packages\ # shared packages |— jodl-tokens\ # design
tokens (colors, typography, spacing, motion) |— jodl-ui\ # shared UI components |—
jodl-motion\ # GSAP + Lenis utilities |— jodl-typography\ # font pairings, reveal
animations |— jodl-hooks\ # shared React hooks |— jodl-patterns\ # reusable pattern
```

```
components | └─ ✓ apps\sarta\ # luxury fashion store (proven, builds OK) └─ X apps\creat-
studio\ STUB - not scaffolded yet └─ X apps\jodlxverse\ STUB - not scaffolded yet
```

10 Current Gaps & Fixes

P1 — BLOCKING (SYSTEM CAN'T RUN WITHOUT THESE)

- Orchestrator system prompts missing** P1

master-orchestrator, design-orchestrator, architecture-orchestrator, implementation-orchestrator, security-orchestrator, ship-orchestrator all fall back to generic one-liner. Without proper system prompts they won't output the correct JSON spawn-tasks block.

→ Create packages/jodl-system/agents/master-orchestrator/system-prompt.md (and each domain orch)
- Gemini model IDs unverified** P1

routing-matrix.yaml uses "gemini-3.5-flash" — this model ID may not exist in the Google AI API. Need to verify against Google AI Studio and update to valid IDs (e.g. "gemini-2.0-flash", "gemini-1.5-flash").

→ Check Google AI model list and update routing-matrix.yaml

P2 — HIGH (LIMITS SYSTEM EFFECTIVENESS)

- Embedding layer missing (sqlite-vss blocked)** P2

The planned Phase 2 vector search uses sqlite-vss which requires C++ build tools (Visual Studio) on Windows. Better alternative: store embeddings in a .jsonl file using OpenAI embedding API. Cosine similarity in pure JS, no native deps.

→ Implement jsonl-based embeddings in search.ts using openai.embeddings.create()
- Pattern registry has 5 components (needs 30+)** P2

The intelligence layer only knows 5 components, all from SARTA. The compose() and search() functions return empty or thin results for anything outside SARTA's pattern set. Every new project adds components but they haven't been curated.

→ Run jodl curate on 5+ reference sites, extract 30+ components into registry
- Parallel group execution not automated** P2

Tasks with the same parallelGroup should fire simultaneously when their shared deps are done. Currently the bus only processes one task at a time per jodl next call. Need a loop mode or daemon watcher.

→ Add jodl work --loop (runs next continuously until no tasks remain)

P3 — MEDIUM (POLISH + RELIABILITY)

- README commands are stale** P3

The monorepo README still references old pnpm run commands from pre-Turbo setup. New commands: pnpm jodl brief/next/status.

→ Update docs/README.md with current command reference
- Knowledge packs missing for most agents** P3

Only uiux-master has a knowledge-pack.md. All other agents rely solely on system-prompt.md. Domain-specific reference docs (CSS patterns, security checklists, deploy runbooks) would dramatically improve output quality.

→ Write knowledge-pack.md for each leaf agent role

creat-studio and jodlxverse apps not scaffolded

P3

apps/creat-studio and apps/jodlxverse are stub directories in the monorepo with no content. They were declared in the workspace but never built out.

→ Scaffold with pnpm create vite or copy from sarta base

P4 — NICE TO HAVE

Web dashboard for session monitoring

P4

Currently jodl watch shows a terminal table. A browser-based dashboard showing the DAG of tasks, live progress, and per-provider status would make the system much more observable.

→ Build a simple Express + SSE server, React dashboard in apps/jodl-dashboard

Cost tracking per session

P4

No token/cost tracking across providers. For production use you want to know how much a session cost across all 3 providers (Claude Opus is expensive, GPT-5 is expensive).

→ Track tokens in done-*.yaml, aggregate with jodl status --cost

11 Production Roadmap

SPRINT 1 Multi-Provider Execution

✓ DONE

- runClaude / runGemini / runOpenAI provider runners in bus-commands.ts
- next() branches on assignedProvider — antigravity + codex auto-execute
- whoami shows all 3 API key statuses
- --force-reclaim to unstick stuck claimed tasks
- domainForRole() + loadSessionContext(domain) — scoped context injection
- @google/genai added to jodl-cli deps; pnpm allowBuilds configured

SPRINT 2 Orchestrator System Prompts + Model Verification

NEXT

- Write system-prompt.md for all 6 orchestrator roles (spawn-tasks JSON format enforcement)
- Verify Gemini model IDs against Google AI API — fix routing-matrix.yaml
- Add jodl work --loop (continuous execution until queue empty)
- Test first fully-automated 3-provider session end-to-end

SPRINT 3 Intelligence Layer & Registry Growth

PLANNED

- Implement embedding index using OpenAI API + JSONL vectors (no native deps)
- Grow registry to 30+ components via jodl curate on 5+ reference sites
- Write knowledge-pack.md for frontend-master, architect, schema-master, backend-master
- Update README with current command reference

SPRINT 4 App Scaffolding + Dashboard

PLANNED

- Scaffold creat-studio app (existing UI at create-studio-beta.vercel.app)
- Scaffold jodlxverse app
- Web dashboard for session DAG + live provider status
- Token/cost tracking per session and per provider

12 How to Run — Operational Runbook

Prerequisite: At least one API key set. Claude requires `ANTHROPIC_API_KEY`. For full multi-provider runs, also set `GOOGLE_API_KEY` and `OPENAI_API_KEY`.

Step 1 — Set your provider identity

```
# In PowerShell (Windows): $env:JODL_PROVIDER = "claude-code" # or: antigravity | codex
$env:ANTHROPIC_API_KEY = "sk-ant-..." $env:GOOGLE_API_KEY = "AIza..." # for Gemini tasks
$env:OPENAI_API_KEY = "sk-..." # for GPT tasks # Verify: pnpm jodl whoami
```

Step 2 — Submit a brief

```
# Any provider can submit a brief: pnpm jodl brief "Build a dashboard for analytics with
charts, filter, export" # Skip research phase for faster prototypes: pnpm jodl brief --skip-
research "Quick prototype: login form" # Output: session ID + master-orchestrator task
created # 📄 Session sess-a1b2c3d4 created # ✓ Master orchestrator task spawned
```

Step 3 — Run your next task (auto)

```
# The bus finds the next pending task for YOUR provider: pnpm jodl next # See what's waiting
without claiming: pnpm jodl next --dry-run # Limit to one session: pnpm jodl next --session-
```

```
id sess-a1b2c3d4 # ✓ Claimed. 🤖 Running claude-code agent (claude-opus-4-7)... # [streams
output to terminal] # ✓ Task done. 🌱 Spawned 5 child task(s).
```

Step 4 — Check queue status

```
pnpm jodl status # Shows all sessions, tasks by status (pending/claimed/done), which
provider owns each pnpm jodl watch # Live terminal dashboard (run in a dedicated terminal
window)
```

Manual flow (for when API auto-run isn't available)

```
# 1. Claim a specific task (prints the brief for you to paste into your AI): pnpm jodl claim
a1b2c3d4 # 2. Copy the printed brief → paste into Claude.ai / Gemini / ChatGPT # 3. Copy the
output to a file: # output.md # 4. Submit the output: pnpm jodl submit a1b2c3d4 -f output.md
# Or paste from stdin: pnpm jodl submit a1b2c3d4 # (paste output, press Ctrl+Z + Enter on
Windows to finish)
```

Emergency recovery

```
# Task stuck in "claimed" (execution died mid-run): pnpm jodl next --force-reclaim a1b2c3d4
# Renames claimed-*-a1b2c3d4.yaml → pending-a1b2c3d4.yaml # Wrong brain entry? Undo it: cd
D:\.agents git log --oneline -10 git revert <sha>
```

Multi-provider parallel session (full power)

```
# Terminal 1 (Claude): $env:JODL_PROVIDER = "claude-code"; pnpm jodl next # Terminal 2
(Gemini — needs GOOGLE_API_KEY set): $env:JODL_PROVIDER = "antigravity"; pnpm jodl next #
Terminal 3 (GPT — needs OPENAI_API_KEY set): $env:JODL_PROVIDER = "codex"; pnpm jodl next #
Terminal 4 (monitor): pnpm jodl watch # Each terminal runs its provider's tasks. Parallel
groups execute simultaneously. # The bus handles races with atomic rename — no coordination
needed.
```